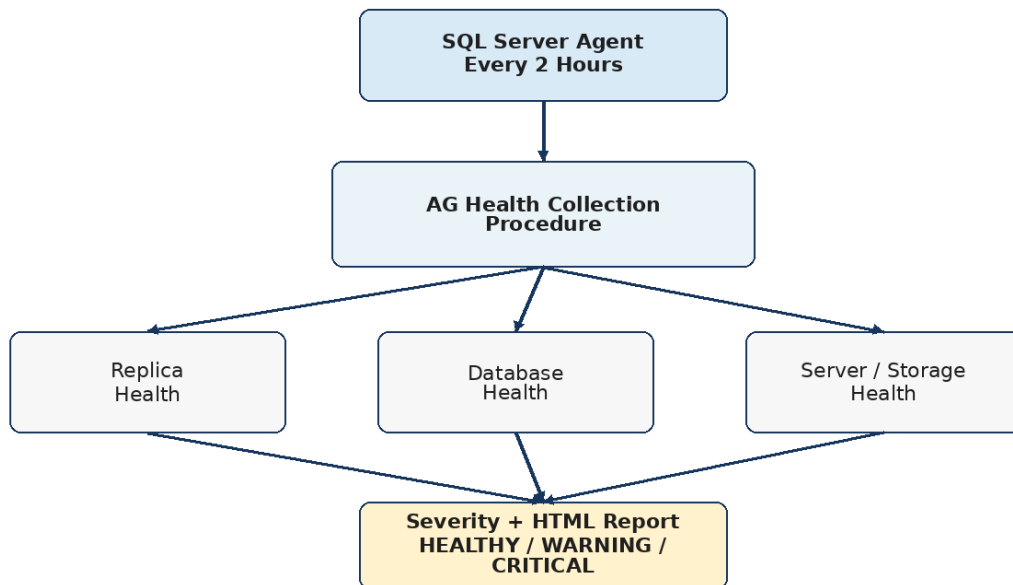


# SQL SERVER ALWAYS ON

## Automated Health Monitoring & Alerting

Production Technical Article and Standard Operating Procedure



### Two-Hour HEALTHY / WARNING / CRITICAL Reporting

Prepared by: Praveen Madupu

Version: 1.0

Date: 11 September 2026

## Document Control

| Item                 | Value                         | Item             | Value               |
|----------------------|-------------------------------|------------------|---------------------|
| <b>Document Type</b> | Technical Article / SOP       | <b>Version</b>   | 1.0                 |
| <b>Platform</b>      | SQL Server Always On AG       | <b>Schedule</b>  | Every 2 hours       |
| <b>Notification</b>  | HTML Database Mail            | <b>Retention</b> | 90 days default     |
| <b>Target</b>        | SQL Server 2019 / 2022 / 2025 | <b>Status</b>    | Production template |

*Scope: This SOP monitors SQL-visible Always On health and supporting SQL Server indicators. It complements, but does not replace, Windows, WSFC, network, storage, hypervisor, and enterprise infrastructure monitoring.*

---

## Table of Contents

| Section                                     | Page | Section                                      | Page |
|---------------------------------------------|------|----------------------------------------------|------|
| 1. Executive Summary                        | 3    | 16. Default Thresholds and Tuning            | 7    |
| 2. Problem Statement                        | 3    | 17. Deployment Prerequisites                 | 8    |
| 3. Monitoring Architecture                  | 3    | 18. Objects Created by the Deployment Script | 8    |
| 4. Always On Data Movement Path             | 4    | 19. Primary and Secondary Email Logic        | 9    |
| 5. Monitoring Components                    | 4    | 20. Post-Deployment Validation               | 9    |
| 6. Health Classification                    | 4    | 21. Troubleshooting Workflow                 | 10   |
| 7. Replica-Level Monitoring                 | 5    | 22. Production Scenarios                     | 10   |
| 8. Database-Level Monitoring                | 5    | 23. RCA Template                             | 11   |
| 9. Send Queue and Transport Analysis        | 6    | 24. Operational Queries                      | 11   |
| 10. Redo Queue and Secondary Apply Analysis | 6    | 25. Security and Permissions                 | 12   |
| 11. Secondary Lag                           | 6    | 26. Limitations and Complementary Monitoring | 12   |
| 12. HADR Endpoint Monitoring                | 6    | 27. Best-Practice Operating Principle        | 12   |
| 13. Disk Space and Transaction Log Reuse    | 6    | 28. Microsoft Official References            | 12   |
| 14. Recent SQL Server ERRORLOG Analysis     | 6    | Appendix A - Complete Deployment Script      | 14   |
| 15. Alert Matrix                            | 7    | Appendix B - Production Deployment Checklist | 38   |

*Page numbers validated against this document version (v1.0).*

## 1. Executive Summary

SQL Server Always On Availability Groups are designed for high availability and disaster recovery, but an enterprise monitoring solution must look beyond a single synchronization-state label. A database can be connected while queues are growing, a secondary can be falling behind because redo is slow, or a transient network problem can recover before the next manual check.

This solution runs every two hours through SQL Server Agent, collects replica, database, endpoint, storage, transaction-log and recent HADR error-log evidence, classifies the result as HEALTHY, WARNING or CRITICAL, sends an HTML report, and stores history for trend analysis and RCA.

- Healthy heartbeat from the primary every two hours.
- Fail-safe WARNING/CRITICAL notification from secondary replicas by default.
- Configurable queue, lag and disk thresholds.
- Historical run, finding and database-replica snapshot tables.
- Complete deployment script included in Appendix A.

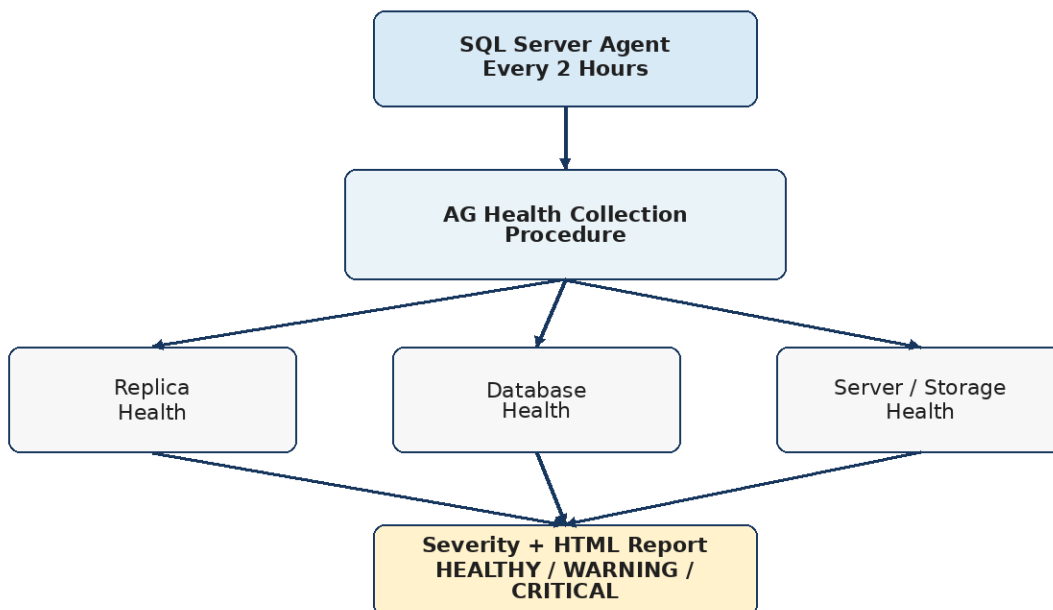
## 2. Problem Statement

A production DBA commonly starts troubleshooting when an availability database shows NOT SYNCHRONIZING. That state is a symptom, not necessarily the root cause. The root cause can exist in SQL Server, the HADR endpoint, network transport, storage, WSFC, or the secondary redo path.

A strong monitoring design should answer four questions quickly:

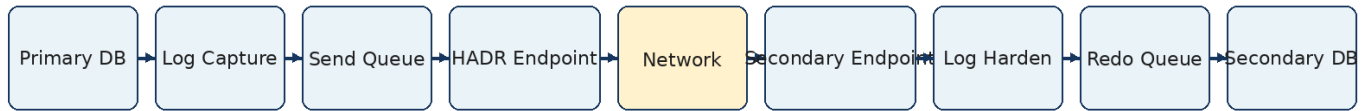
1. What is unhealthy?
2. Which availability group, database, or replica is affected?
3. How severe is the condition?
4. What should the DBA investigate first?

## 3. Monitoring Architecture



The job is deployed on each SQL Server instance that participates in an Availability Group. The primary replica sends the routine two-hour heartbeat. Secondary-only instances suppress duplicate healthy messages but still send local WARNING or CRITICAL findings unless configured otherwise.

## 4. Always On Data Movement Path



Monitoring should cover the full path from primary transaction log generation through transport, hardening and redo. A failure or slowdown in any stage can appear later as synchronization lag or NOT SYNCHRONIZING.

## 5. Monitoring Components

| Component               | Purpose                                                        | Primary SQL Object                                      |
|-------------------------|----------------------------------------------------------------|---------------------------------------------------------|
| Replica health          | Role, operational state, connection and synchronization health | sys.dm_hadr_availability_replica_states                 |
| Database replica health | Synchronization, suspend state, send/redo queues and rates     | sys.dm_hadr_database_replica_states                     |
| AG metadata             | AG and replica configuration                                   | sys.availability_groups /<br>sys.availability_replicas  |
| Endpoint health         | HADR endpoint state and TCP port                               | sys.database_mirroring_endpoints /<br>sys.tcp_endpoints |
| Storage health          | Free space on SQL data/log volumes                             | sys.dm_os_volume_stats                                  |
| Transaction log         | Detect AVAILABILITY_REPLICA log reuse waits                    | sys.databases                                           |
| Recent HADR evidence    | Search current SQL Server ERRORLOG                             | sys.sp_readerrorlog                                     |
| Notification            | HTML email delivery                                            | msdb.dbo.sp_send_dbmail                                 |
| Scheduling              | Two-hour recurring execution                                   | SQL Server Agent                                        |

## 6. Health Classification

| Status   | Meaning                                                  | Typical Examples                                                                                                                                                        |
|----------|----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HEALTHY  | No configured warning or critical condition detected.    | Connected replicas; target synchronization health; queues/lag below thresholds; endpoint started; acceptable free space.                                                |
| WARNING  | AG is operating but needs investigation or trend review. | PARTIALLY_HEALTHY; elevated queue; elevated lag; low disk; recent connection error; AVAILABILITY_REPLICA log reuse wait.                                                |
| CRITICAL | Immediate HA/DR or database-health risk.                 | DISCONNECTED; NOT_HEALTHY; NOT SYNCHRONIZING; suspended data movement; failed/no quorum; stopped endpoint; suspect/recovery-pending database; severe lag/disk pressure. |

## 7. Replica-Level Monitoring

### 7.1 Connection State

The replica DMV exposes CONNECTED and DISCONNECTED. Microsoft documents that the primary tracks every secondary connection, while a secondary tracks its connection to the primary. DISCONNECTED is treated as CRITICAL because the transport path is unavailable.

- Check SQL Server service on the remote replica.
- Verify DNS and TCP connectivity.
- Validate HADR endpoint state and port.
- Check firewall rules and endpoint authentication.
- Correlate with SQL Server ERRORLOG and Windows/WSFC events.

### 7.2 Operational State

The procedure treats FAILED, FAILED\_NO\_QUORUM and OFFLINE as CRITICAL. These states can point to cluster/quorum, service, or replica operational failures and should be correlated with WSFC health.

### 7.3 Synchronization Health

Synchronization health is preferable to a simplistic rule that requires every secondary database to display SYNCHRONIZED. Microsoft defines target synchronization by commit mode: synchronous-commit replicas should be synchronized, while asynchronous-commit replicas can be healthy while synchronizing.

## 8. Database-Level Monitoring

### 8.1 NOT SYNCHRONIZING

NOT SYNCHRONIZING is treated as CRITICAL. The monitoring email adds related evidence such as replica connectivity, suspend state, queues, endpoint state and recent HADR errors so the DBA can identify the failing layer.

### 8.2 Suspended Data Movement

When `is_suspended = 1`, the procedure captures `suspend_reason_desc` and raises a CRITICAL finding. The DBA should investigate the reason before issuing SET HADR RESUME; blindly resuming can hide the symptom without correcting the cause.

### 8.3 Database State

RECOVERY\_PENDING, SUSPECT, EMERGENCY and OFFLINE are treated as CRITICAL database-health findings. These require database recovery, storage/I/O and SQL ERRORLOG investigation, not only AG troubleshooting.

## 9. Send Queue and Transport Analysis

Microsoft reports `log_send_queue_size` in KB and `log_send_rate` in KB/sec. A large queue is important, but the rate and trend determine how serious it is. A 10 GB queue draining at 100 MB/sec can be less risky than a 2 GB queue sending at 100 KB/sec.

Estimated send drain time is calculated as:

$\text{Estimated Send Delay Seconds} = \text{log\_send\_queue\_size} / \text{NULLIF}(\text{log\_send\_rate}, 0)$

This is an operational estimate, not a guaranteed recovery time. Workload generation and throughput can change continuously.

## 10. Redo Queue and Secondary Apply Analysis

redo\_queue\_size represents received log records not yet redone on the secondary, while redo\_rate represents the average redo rate. A growing redo queue with healthy transport usually shifts attention toward secondary storage, CPU, read workload, or redo pressure.

Estimated Redo Delay Seconds = redo\_queue\_size / NULLIF(redo\_rate, 0)

## 11. Secondary Lag

Where secondary\_lag\_seconds is available, the framework uses it as another signal of replica delay. The default warning and critical values are operational policy values and must be tuned to the business RPO/RTO, workload, network topology and commit mode.

## 12. HADR Endpoint Monitoring

Always On replica communication uses the database mirroring endpoint. The procedure validates that an endpoint exists and that its state is STARTED, while also reporting the configured TCP port. A stopped or missing endpoint is classified as CRITICAL on an Always On enabled replica.

## 13. Disk Space and Transaction Log Reuse

### 13.1 SQL Volume Free Space

The procedure evaluates the volumes hosting SQL Server database files using sys.dm\_os\_volume\_stats. Default warning and critical percentages are starting values only. Multi-terabyte estates may also require absolute free-GB thresholds in enterprise monitoring.

### 13.2 AVAILABILITY\_REPLICA Log Reuse Wait

A log\_reuse\_wait\_desc value of AVAILABILITY\_REPLICA is reported as a WARNING because a lagging, disconnected or suspended replica can prevent transaction-log truncation and contribute to log growth.

## 14. Recent SQL Server ERRORLOG Analysis

Current DMVs can look healthy after an intermittent failure has already recovered. The framework therefore reads the current SQL Server error log and looks back over the configured period for HADR/availability entries that contain failure-oriented terms such as error, fail, disconnect, timeout, suspend, lease, not synchronizing or terminated.

Microsoft documents sys.sp\_readerrorlog as the supported stored procedure for reading and filtering SQL Server or SQL Server Agent error logs. SQL Server 2019 and earlier require VIEW SERVER STATE; SQL Server 2022 and later require VIEW ANY ERROR LOG or VIEW SERVER PERFORMANCE STATE for this operation.

## 15. Alert Matrix

| Area        | Condition                           | Severity | DBA First Action                                                     |
|-------------|-------------------------------------|----------|----------------------------------------------------------------------|
| Replica     | DISCONNECTED                        | CRITICAL | Check service, endpoint, DNS, firewall and network path.             |
| Replica     | FAILED / FAILED_NO_QUORUM / OFFLINE | CRITICAL | Check WSFC/quorum, service and cluster events.                       |
| Sync health | NOT_HEALTHY                         | CRITICAL | Identify affected DB/replica and related transport/apply symptoms.   |
| Sync health | PARTIALLY_HEALTHY                   | WARNING  | Review target synchronization state and queues.                      |
| Database    | NOT SYNCHRONIZING                   | CRITICAL | Check connectivity, suspend state, transport, endpoint and ERRORLOG. |
| Data        | Suspended                           | CRITICAL | Investigate suspend reason before resume.                            |

| Area            | Condition                                        | Severity           | DBA First Action                                          |
|-----------------|--------------------------------------------------|--------------------|-----------------------------------------------------------|
| movement        |                                                  |                    |                                                           |
| Send queue      | Above warning/critical threshold                 | WARNING / CRITICAL | Compare queue trend with send rate.                       |
| Redo queue      | Above warning/critical threshold                 | WARNING / CRITICAL | Check secondary I/O, CPU and read workload.               |
| Secondary lag   | Above warning/critical threshold                 | WARNING / CRITICAL | Correlate send, harden and redo performance.              |
| Endpoint        | Not STARTED                                      | CRITICAL           | Check endpoint config/authentication/port/firewall.       |
| Database state  | SUSPECT / RECOVERY_PENDING / EMERGENCY / OFFLINE | CRITICAL           | Investigate recovery and storage immediately.             |
| Disk            | <=15% / <=5% free by default                     | WARNING / CRITICAL | Free or extend storage; validate growth requirements.     |
| Log reuse       | AVAILABILITY_REPLICA                             | WARNING            | Find lagging/disconnected/suspended replica.              |
| Recent ERRORLOG | HADR failure-oriented entry                      | WARNING / CRITICAL | Review surrounding error-log and infrastructure evidence. |

## 16. Default Thresholds and Tuning

| Metric                     | Warning | Critical | Notes                                                 |
|----------------------------|---------|----------|-------------------------------------------------------|
| Log Send Queue             | 512 MB  | 2 GB     | Starting values; tune by workload and transport rate. |
| Redo Queue                 | 512 MB  | 2 GB     | Starting values; tune by secondary capacity.          |
| Estimated queue drain time | 300 sec | 900 sec  | Derived from queue/rate; variable by workload.        |
| Secondary lag              | 300 sec | 900 sec  | Align with RPO/RTO and topology.                      |
| Disk free space            | <=15%   | <=5%     | Add absolute GB thresholds in very large estates.     |
| ERRORLOG lookback          | 2 hours | N/A      | Matches monitoring cadence.                           |
| History retention          | 90 days | N/A      | Adjust to operational/compliance policy.              |

*Important: These queue, lag and disk values are operational defaults in this implementation. They are not universal Microsoft limits.*

## 17. Deployment Prerequisites

5. Always On Availability Groups enabled on the SQL Server instance.
6. SQL Server Agent running.
7. Database Mail configured and a working mail profile available.
8. Execution context has the required DMV/server-state permissions for the SQL Server version.
9. Execution context can read the SQL Server ERRORLOG as required by version.
10. Change MailProfile and Recipients in dbo.AGMonitorConfig after deployment or in the script before rollout.
11. Deploy the same monitoring objects on every AG replica for fail-safe coverage.

## 18. Objects Created by the Deployment Script

| Object                        | Type     | Purpose                                                     |
|-------------------------------|----------|-------------------------------------------------------------|
| DBA_Admin                     | Database | Dedicated monitoring database.                              |
| dbo.AGMonitorConfig           | Table    | Thresholds, mail profile, recipient and retention settings. |
| dbo.AGMonitorRunHistory       | Table    | Each procedure execution and email outcome.                 |
| dbo.AGMonitorFindingHistory   | Table    | Historical warning/critical/healthy findings.               |
| dbo.AGMonitorDatabaseSnapshot | Table    | Queue, rate, lag and replica-state snapshots.               |

| Object                                       | Type             | Purpose                                                                |
|----------------------------------------------|------------------|------------------------------------------------------------------------|
| dbo.ufn_AGMonitor_HtmlEncode                 | Function         | Safe HTML encoding for email content.                                  |
| dbo.usp_AG_HealthCheck_Email                 | Stored procedure | Collects health, classifies severity, persists history and sends mail. |
| dbo.vw_AGMonitorLatestRuns                   | View             | Latest monitoring executions.                                          |
| dbo.vw_AGMonitorRecentProblems               | View             | Recent WARNING/CRITICAL findings.                                      |
| DBA - Always On Health Alert - Every 2 Hours | SQL Agent Job    | Scheduled health collection and notification.                          |

## 19. Primary and Secondary Email Logic

The default behavior minimizes duplicate healthy emails without sacrificing fail-safe coverage:

PRIMARY: HEALTHY -> Email | WARNING -> Email | CRITICAL -> Email

SECONDARY: HEALTHY -> Suppressed | WARNING -> Email | CRITICAL -> Email

Set SendHealthyFromSecondary = 1 only when the operations team explicitly wants a healthy heartbeat from every replica every two hours.

## 20. Post-Deployment Validation

12. Update dbo.AGMonitorConfig with the real Database Mail profile and recipients.
13. Execute the procedure with @ForceEmail = 1 and @Debug = 1 on each replica.
14. Confirm HTML email receipt and formatting.
15. Confirm dbo.AGMonitorRunHistory shows the execution.
16. Review dbo.AGMonitorFindingHistory and dbo.AGMonitorDatabaseSnapshot.
17. Verify the SQL Agent job is enabled and scheduled every two hours.
18. Test a controlled non-production failure such as a stopped HADR endpoint or suspended data movement, then confirm detection and restore the environment.

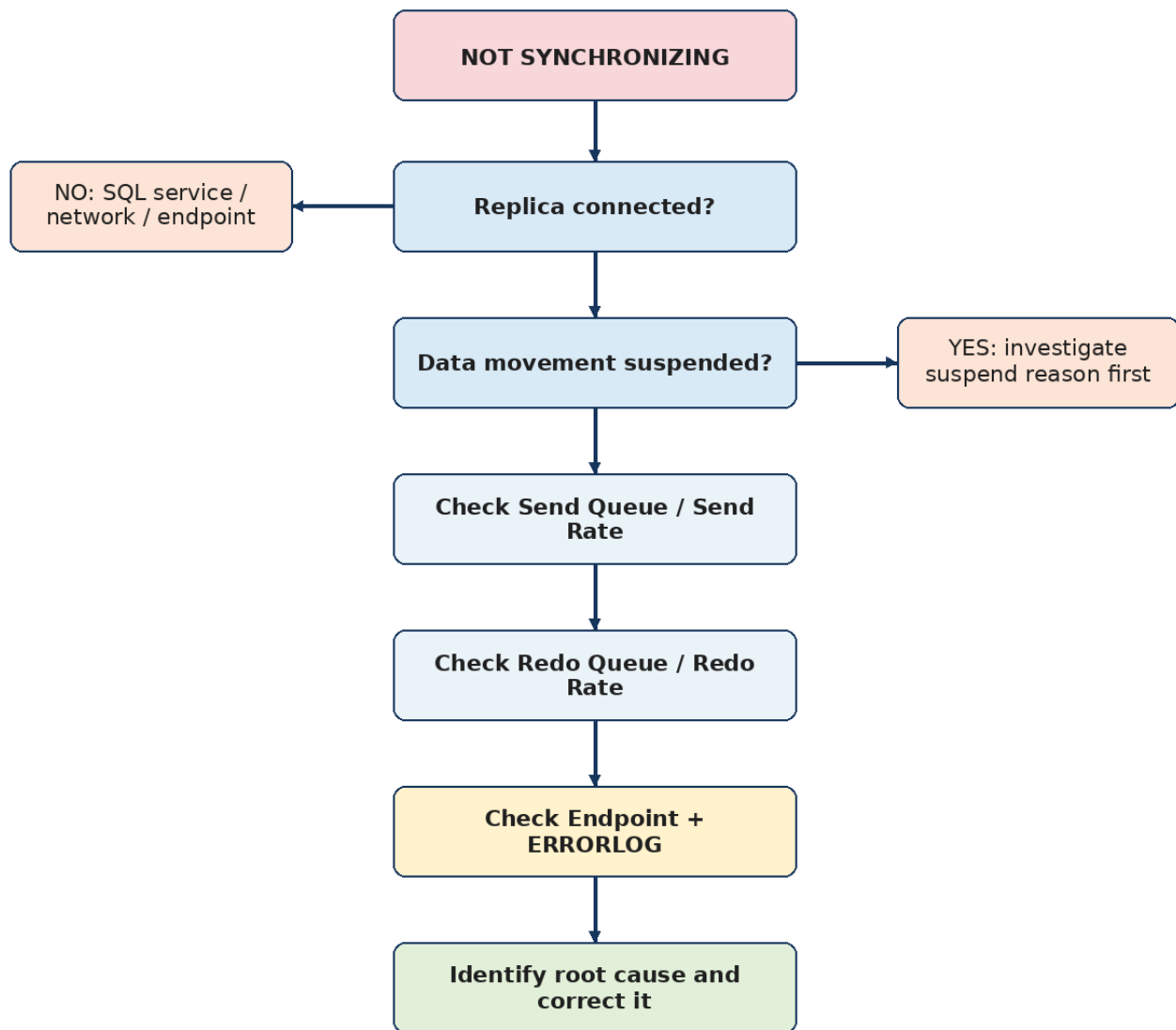
### 20.1 Forced Test

```
EXEC DBA_Admin.dbo.usp_AG_HealthCheck_Email @ForceEmail = 1, @Debug = 1;
```

### 20.2 Normal Production Test

```
EXEC DBA_Admin.dbo.usp_AG_HealthCheck_Email @ForceEmail = 0, @Debug = 1;
```

## 21. Troubleshooting Workflow for NOT SYNCHRONIZING



Do not restart services as the first response. Capture current evidence first, identify the failing layer, correct the cause, and then watch synchronization recover.

## 22. Production Scenarios

### 22.1 Network or Endpoint Failure

Typical pattern: replica DISCONNECTED, NOT\_HEALTHY, database NOT SYNCHRONIZING, send queue increasing and send rate zero. This points toward transport rather than redo. Check SQL service, HADR endpoint, TCP port, DNS, firewall and network path.

### 22.2 Slow Secondary Storage

Typical pattern: replica CONNECTED, send queue small, redo queue growing, redo rate low and secondary lag increasing. Focus on secondary storage latency/throughput, CPU, read workload and redo processing.

## 22.3 Primary Log Generation Exceeds Transport

Typical pattern: send queue increases while redo queue remains small. Compare primary log generation with send throughput, network capacity and remote hardening performance.

## 22.4 Intermittent Failure That Self-Recovers

Current DMV state may be healthy when the two-hour job runs, but recent HADR ERRORLOG entries can reveal a disconnect or timeout during the lookback window. This is why current state and recent evidence should be correlated.

## 22.5 Disk Capacity Pressure

A critical free-space finding combined with queue growth or lag can indicate storage capacity/performance pressure. Resolve space risk before autogrowth, log growth or recovery operations are affected.

# 23. RCA Template

| RCA Field         | What to Record                                                                                   |
|-------------------|--------------------------------------------------------------------------------------------------|
| Incident time     | Start, detection, recovery and closure time.                                                     |
| Affected scope    | AG, database, primary/secondary replicas, listener/application impact.                           |
| Initial symptom   | NOT SYNCHRONIZING, disconnect, queue growth, suspended movement, etc.                            |
| Evidence          | DMV values, ERRORLOG entries, WSFC/network/storage evidence.                                     |
| Root cause        | Confirmed technical cause; avoid listing only the symptom.                                       |
| Corrective action | What restored service or health.                                                                 |
| Validation        | How synchronization, queues, lag, endpoint and application health were verified.                 |
| Preventive action | Threshold change, capacity change, network fix, patch, procedure update, automation improvement. |
| Owner / due date  | Person/team responsible for follow-up action.                                                    |

# 24. Operational Queries

## 24.1 Latest Monitor Runs

```
SELECT * FROM DBA_Admin.dbo.vw_AGMonitorLatestRuns;
```

## 24.2 Recent Problems

```
SELECT TOP (100) *
FROM DBA_Admin.dbo.vw_AGMonitorRecentProblems
ORDER BY CaptureTime DESC;
```

## 24.3 24-Hour Queue and Lag Trend

```
SELECT CaptureTime, AGName, DatabaseName, ReplicaName,
    LogSendQueueKB / 1024.0 AS LogSendQueueMB, LogSendRateKBps, EstimatedSendDelaySeconds,
    RedoQueueKB / 1024.0 AS RedoQueueMB, RedoRateKBps, EstimatedRedoDelaySeconds, SecondaryLagSeconds
FROM DBA_Admin.dbo.AGMonitorDatabaseSnapshot
WHERE CaptureTime >= DATEADD(HOUR,-24,SYSDATETIME())
ORDER BY CaptureTime DESC, AGName, DatabaseName, ReplicaName;
```

# 25. Security and Permissions

Use least privilege and test with the real SQL Server Agent execution context. Microsoft documents VIEW SERVER STATE requirements for many HADR DMVs on SQL Server 2019 and earlier, while SQL Server 2022 and later generally use VIEW SERVER PERFORMANCE STATE for these DMVs. sys.sp\_readerrorlog has version-specific permissions: VIEW SERVER STATE for SQL Server 2019 and earlier, and VIEW ANY ERROR LOG or VIEW SERVER PERFORMANCE STATE for SQL Server 2022 and later.

- Do not grant sysadmin only to make monitoring work unless organizational policy explicitly requires it.

- Protect DBA\_Admin from unnecessary application access.
- Restrict modification rights on the config table, procedure and Agent job.
- Treat email recipient lists and SMTP configuration as production configuration data.

## 26. Limitations and Complementary Monitoring

A SQL Server health procedure can detect symptoms of external failures but cannot directly prove every physical cause. For example, a failed switch, SAN controller, hypervisor or WAN circuit can appear inside SQL Server as a disconnect, timeout, stalled transport or storage symptom.

- Windows and OS monitoring
- WSFC cluster and quorum monitoring
- Network device and WAN monitoring
- Storage/SAN monitoring
- VM/hypervisor monitoring
- SMTP/Database Mail monitoring
- Enterprise observability and incident-management integration

## 27. Best-Practice Operating Principle

*Quick root-cause identification is more valuable than blindly restarting services. Detect -> Correlate -> Identify Root Cause -> Correct -> Validate -> Trend.*

---

## 28. Microsoft Official References

- **sys.dm\_hadr\_availability\_replica\_states (Transact-SQL)**  
<https://learn.microsoft.com/en-us/sql/relational-databases/system-dynamic-management-views/sys-dm-hadr-availability-replica-states-transact-sql?view=sql-server-ver17>
- **sys.dm\_hadr\_database\_replica\_states (Transact-SQL)**  
<https://learn.microsoft.com/en-us/sql/relational-databases/system-dynamic-management-objects/sys-dm-hadr-database-replica-states-transact-sql?view=sql-server-ver17>
- **sys.dm\_hadr\_availability\_group\_states (Transact-SQL)**  
<https://learn.microsoft.com/en-us/sql/relational-databases/system-dynamic-management-objects/sys-dm-hadr-availability-group-states-transact-sql?view=sql-server-ver17>
- **Troubleshooting long send queueing in an Always On availability group**  
<https://learn.microsoft.com/en-us/troubleshoot/sql/database-engine/availability-groups/troubleshooting-log-send-queueing-in-alwayson-availability-group>
- **sys.sp\_readerrorlog (Transact-SQL)**  
<https://learn.microsoft.com/en-us/sql/relational-databases/system-stored-procedures/sp-readerrorlog-transact-sql?view=sql-server-ver17>
- **Database Mail stored procedures**  
<https://learn.microsoft.com/en-us/sql/relational-databases/system-stored-procedures/database-mail-stored-procedures-transact-sql?view=sql-server-ver17>
- **Create and Attach Schedules to Jobs**  
<https://learn.microsoft.com/en-us/ssms/agent/create-and-attach-schedules-to-jobs>
- **Monitor Availability Groups**  
<https://learn.microsoft.com/en-us/sql/database-engine/availability-groups/windows/monitoring-of-availability-groups-sql-server?view=sql-server-ver17>

## Appendix A - Complete Production Deployment Script

The following script is the complete implementation generated for this SOP. Review and change the mail profile and recipients before production deployment. The script is intentionally included in full.

```
/*
=====
SQL Server Always On Availability Groups - Production Health Monitoring
Version   : 1.0
Target    : SQL Server 2019 / 2022 / 2025
Schedule  : Every 2 hours
Purpose   : Healthy heartbeat + WARNING/CRITICAL Always On notifications
Deployment : Run on EACH SQL Server instance participating in an AG
=====

IMPORTANT BEFORE RUNNING
-----
1. Configure Database Mail first.
2. In section "CONFIGURATION VALUES", set:
    MailProfile
    Recipients
3. Deploy on every AG replica.
4. SQL Server Agent must be running.
5. Recommended permissions for the Agent job owner/execution context:
    VIEW SERVER STATE (SQL Server 2019 and earlier)
    VIEW SERVER PERFORMANCE STATE as applicable on newer versions
    VIEW ANY ERROR LOG or equivalent on SQL Server 2022+
6. Queue/disk thresholds below are STARTING VALUES. Tune them for your workload.

MAIL BEHAVIOR
-----
Default = PRIMARY_PLUS_FAILSAFE

* A server hosting a PRIMARY replica sends the normal 2-hour heartbeat:
    HEALTHY / WARNING / CRITICAL

* A server hosting only SECONDARY replicas sends mail only when it detects
    WARNING/CRITICAL locally. This prevents duplicate healthy emails while still
    providing fail-safe notification if the primary is unavailable.

To receive an email from EVERY replica every two hours, change:
    SendHealthyFromSecondary = 1
=====
*/

SET NOCOUNT ON;
SET XACT_ABORT ON;
GO

/*=====
0. PRECHECK
=====*/

IF CAST(SERVERPROPERTY('IsHadrEnabled') AS int) <> 1
BEGIN
    THROW 51000, 'Always On Availability Groups is not enabled on this SQL Server instance.', 1;
END;
GO
```

```

/*=====
1. DBA DATABASE
=====*/

USE master;
GO

IF DB_ID(N'DBA_Admin') IS NULL
BEGIN
    PRINT 'Creating DBA_Admin database...';
    CREATE DATABASE DBA_Admin;
END;
GO

ALTER DATABASE DBA_Admin SET RECOVERY SIMPLE;
GO

USE DBA_Admin;
GO

/*=====
2. CONFIGURATION TABLE
=====*/

IF OBJECT_ID(N'dbo.AGMonitorConfig', N'U') IS NULL
BEGIN
    CREATE TABLE dbo.AGMonitorConfig
    (
        ConfigID          tinyint    NOT NULL
        CONSTRAINT PK_AGMonitorConfig PRIMARY KEY
        CONSTRAINT CK_AGMonitorConfig_OneRow CHECK (ConfigID = 1),

        Enabled           bit        NOT NULL
        CONSTRAINT DF_AGMonitorConfig_Enabled DEFAULT (1),

        MailProfile        sysname    NOT NULL,
        Recipients         varchar(2000) NOT NULL,

        SendHealthyFromSecondary bit    NOT NULL
        CONSTRAINT DF_AGMonitorConfig_SendHealthySecondary DEFAULT (0),

        SendQueueWarningMB    bigint    NOT NULL
        CONSTRAINT DF_AGMonitorConfig_SendQueueWarning DEFAULT (512),

        SendQueueCriticalMB   bigint    NOT NULL
        CONSTRAINT DF_AGMonitorConfig_SendQueueCritical DEFAULT (2048),

        RedoQueueWarningMB    bigint    NOT NULL
        CONSTRAINT DF_AGMonitorConfig_RedoQueueWarning DEFAULT (512),

        RedoQueueCriticalMB   bigint    NOT NULL
        CONSTRAINT DF_AGMonitorConfig_RedoQueueCritical DEFAULT (2048),

        EstimatedDelayWarningSeconds int    NOT NULL
        CONSTRAINT DF_AGMonitorConfig_DelayWarning DEFAULT (300),

        EstimatedDelayCriticalSeconds int    NOT NULL
        CONSTRAINT DF_AGMonitorConfig_DelayCritical DEFAULT (900),

        SecondaryLagWarningSeconds int    NOT NULL
        CONSTRAINT DF_AGMonitorConfig_LagWarning DEFAULT (300),
    )

```

```

SecondaryLagCriticalSeconds int NOT NULL
CONSTRAINT DF_AGMonitorConfig_LagCritical DEFAULT (900),

DiskWarningPercent decimal(5,2) NOT NULL
CONSTRAINT DF_AGMonitorConfig_DiskWarning DEFAULT (15.00),

DiskCriticalPercent decimal(5,2) NOT NULL
CONSTRAINT DF_AGMonitorConfig_DiskCritical DEFAULT (5.00),

ErrorLogLookbackHours int NOT NULL
CONSTRAINT DF_AGMonitorConfig_ErrorLookback DEFAULT (2),

HistoryRetentionDays int NOT NULL
CONSTRAINT DF_AGMonitorConfig_Retention DEFAULT (90),

LastUpdated datetime2(0) NOT NULL
CONSTRAINT DF_AGMonitorConfig_LastUpdated DEFAULT (SYSDATETIME())
);
END;
GO

/*=====
CONFIGURATION VALUES - CHANGE THESE TWO VALUES
=====*/

IF NOT EXISTS (SELECT 1 FROM dbo.AGMonitorConfig WHERE ConfigID = 1)
BEGIN
    INSERT dbo.AGMonitorConfig
    (
        ConfigID,
        MailProfile,
        Recipients
    )
VALUES
    (
        1,
        N'DBA_Mail_Profile', -- <<< CHANGE
        'DBATeam@yourcompany.com' -- <<< CHANGE
    );
END;
GO

```

/\*  
Example configuration update:

```

UPDATE dbo.AGMonitorConfig
SET MailProfile = N'SQL_DBA_Mail',
    Recipients = 'dba-team@company.com;noc@company.com',
    SendHealthyFromSecondary = 0,
    SendQueueWarningMB = 512,
    SendQueueCriticalMB = 2048,
    RedoQueueWarningMB = 512,
    RedoQueueCriticalMB = 2048,
    EstimatedDelayWarningSeconds = 300,
    EstimatedDelayCriticalSeconds = 900,
    SecondaryLagWarningSeconds = 300,
    SecondaryLagCriticalSeconds = 900,
    DiskWarningPercent = 15,
    DiskCriticalPercent = 5,
    ErrorLogLookbackHours = 2,

```

```

    HistoryRetentionDays    = 90,
    LastUpdated             = SYSDATETIME()
WHERE ConfigID = 1;
GO
*/

/*=====
3. EXECUTION HISTORY
=====*/

IF OBJECT_ID(N'dbo.AGMonitorRunHistory', N'U') IS NULL
BEGIN
    CREATE TABLE dbo.AGMonitorRunHistory
    (
        RunID            bigint IDENTITY(1,1) NOT NULL
            CONSTRAINT PK_AGMonitorRunHistory PRIMARY KEY,
        ServerName       sysname           NOT NULL,
        HostName         sysname           NULL,
        RunStartTime     datetime2(0)      NOT NULL,
        RunEndTime       datetime2(0)      NULL,
        OverallStatus    varchar(10)       NULL,
        CriticalCount    int               NOT NULL DEFAULT (0),
        WarningCount     int               NOT NULL DEFAULT (0),
        HealthyCount     int               NOT NULL DEFAULT (0),
        EmailAttempted   bit               NOT NULL DEFAULT (0),
        EmailQueued      bit               NOT NULL DEFAULT (0),
        EmailError       nvarchar(4000)    NULL,
        ProcedureError   nvarchar(4000)    NULL
    );

    CREATE INDEX IX_AGMonitorRunHistory_RunStartTime
    ON dbo.AGMonitorRunHistory(RunStartTime);
END;
GO

/*=====
4. FINDING HISTORY
=====*/

IF OBJECT_ID(N'dbo.AGMonitorFindingHistory', N'U') IS NULL
BEGIN
    CREATE TABLE dbo.AGMonitorFindingHistory
    (
        FindingID       bigint IDENTITY(1,1) NOT NULL
            CONSTRAINT PK_AGMonitorFindingHistory PRIMARY KEY,
        RunID           bigint              NOT NULL,
        CaptureTime     datetime2(0)        NOT NULL,
        Severity        varchar(10)         NOT NULL,
        Category        varchar(60)         NOT NULL,
        AGName          sysname             NULL,
        DatabaseName    sysname             NULL,
        ReplicaName     sysname             NULL,
        Observation      nvarchar(2048)     NOT NULL,
        Recommendation   nvarchar(2048)     NULL,

        CONSTRAINT FK_AGMonitorFindingHistory_Run
        FOREIGN KEY (RunID)
        REFERENCES dbo.AGMonitorRunHistory(RunID)
    );

    CREATE INDEX IX_AGMonitorFindingHistory_RunID

```

```

    ON dbo.AGMonitorFindingHistory(RunID);

CREATE INDEX IX_AGMonitorFindingHistory_CaptureTime
    ON dbo.AGMonitorFindingHistory(CaptureTime);
END;
GO

/*=====
5. DATABASE/REPLICA SNAPSHOT HISTORY
=====*/

IF OBJECT_ID(N'dbo.AGMonitorDatabaseSnapshot', N'U') IS NULL
BEGIN
    CREATE TABLE dbo.AGMonitorDatabaseSnapshot
    (
        SnapshotID          bigint IDENTITY(1,1) NOT NULL
        CONSTRAINT PK_AGMonitorDatabaseSnapshot PRIMARY KEY,
        RunID                bigint             NOT NULL,
        CaptureTime          datetime2(0)       NOT NULL,
        AGName                sysname            NULL,
        IsDistributed         bit                NULL,
        DatabaseName          sysname            NULL,
        ReplicaName           sysname            NULL,
        IsLocal               bit                NULL,
        RoleDesc              nvarchar(60)       NULL,
        AvailabilityModeDesc  nvarchar(60)       NULL,
        ConnectedStateDesc    nvarchar(60)       NULL,
        SynchronizationStateDesc nvarchar(60)    NULL,
        SynchronizationHealthDesc nvarchar(60)   NULL,
        DatabaseStateDesc     nvarchar(60)       NULL,
        IsSuspended           bit                NULL,
        SuspendReasonDesc     nvarchar(60)       NULL,
        LogSendQueueKB        bigint             NULL,
        LogSendRateKBps       bigint             NULL,
        EstimatedSendDelaySeconds decimal(18,2)   NULL,
        RedoQueueKB           bigint             NULL,
        RedoRateKBps          bigint             NULL,
        EstimatedRedoDelaySeconds decimal(18,2)   NULL,
        SecondaryLagSeconds   bigint             NULL,
        LastSentTime          datetime           NULL,
        LastReceivedTime      datetime           NULL,
        LastHardenedTime      datetime           NULL,
        LastRedoneTime        datetime           NULL,
        LastCommitTime        datetime           NULL,

        CONSTRAINT FK_AGMonitorDatabaseSnapshot_Run
        FOREIGN KEY (RunID)
        REFERENCES dbo.AGMonitorRunHistory(RunID)
    );

CREATE INDEX IX_AGMonitorDatabaseSnapshot_Capture
    ON dbo.AGMonitorDatabaseSnapshot(CaptureTime, AGName, DatabaseName);
END;
GO

/*=====
6. HTML ENCODING FUNCTION
=====*/

CREATE OR ALTER FUNCTION dbo.ufn_AGMonitor_HtmlEncode
(

```

```

    @Input nvarchar(max)
)
RETURNS nvarchar(max)
AS
BEGIN
    IF @Input IS NULL RETURN N'';

    RETURN
        REPLACE(
            REPLACE(
                REPLACE(
                    REPLACE(
                        REPLACE(@Input,
                            N'&', N'&amp;'),
                        N'<', N'&lt;'),
                    N'>', N'&gt;'),
                N''', N'&quot;'),
            N''', N'&#39;');
END;
GO

/*=====
7. MAIN HEALTH CHECK PROCEDURE
=====*/

CREATE OR ALTER PROCEDURE dbo.usp_AG_HealthCheck_Email
    @ForceEmail bit = 0,
    @Debug    bit = 0
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT OFF;

    DECLARE
        @RunID          bigint,
        @RunStart        datetime2(0) = SYSDATETIME(),
        @ServerName      sysname = CAST(SERVERPROPERTY('ServerName') AS sysname),
        @HostName        sysname = CAST(SERVERPROPERTY('MachineName') AS sysname),
        @MailProfile      sysname,
        @Recipients       varchar(2000),
        @Enabled          bit,
        @SendHealthyFromSecondary bit,
        @SendQueueWarningMB    bigint,
        @SendQueueCriticalMB    bigint,
        @RedoQueueWarningMB    bigint,
        @RedoQueueCriticalMB    bigint,
        @DelayWarningSec        int,
        @DelayCriticalSec        int,
        @LagWarningSec          int,
        @LagCriticalSec          int,
        @DiskWarningPct          decimal(5,2),
        @DiskCriticalPct          decimal(5,2),
        @ErrorLookbackHours      int,
        @RetentionDays           int,
        @HasPrimary              bit = 0,
        @CriticalCount           int = 0,
        @WarningCount            int = 0,
        @HealthyCount            int = 0,
        @OverallStatus           varchar(10),
        @ShouldSend              bit = 0,
        @EmailQueued             bit = 0,

```

```

@Subject          nvarchar(255),
@Body             nvarchar(max),
@Rows             nvarchar(max),
@Now              datetime2(0) = SYSDATETIME();

SELECT
  @Enabled          = Enabled,
  @MailProfile      = MailProfile,
  @Recipients       = Recipients,
  @SendHealthyFromSecondary = SendHealthyFromSecondary,
  @SendQueueWarningMB = SendQueueWarningMB,
  @SendQueueCriticalMB = SendQueueCriticalMB,
  @RedoQueueWarningMB = RedoQueueWarningMB,
  @RedoQueueCriticalMB = RedoQueueCriticalMB,
  @DelayWarningSec   = EstimatedDelayWarningSeconds,
  @DelayCriticalSec   = EstimatedDelayCriticalSeconds,
  @LagWarningSec      = SecondaryLagWarningSeconds,
  @LagCriticalSec     = SecondaryLagCriticalSeconds,
  @DiskWarningPct     = DiskWarningPercent,
  @DiskCriticalPct    = DiskCriticalPercent,
  @ErrorLookbackHours = ErrorLogLookbackHours,
  @RetentionDays      = HistoryRetentionDays
FROM dbo.AGMonitorConfig
WHERE ConfigID = 1;

IF @Enabled IS NULL
  THROW 51001, 'AG monitor configuration row ConfigID=1 does not exist.', 1;

IF @Enabled = 0
BEGIN
  PRINT 'AG monitoring is disabled in dbo.AGMonitorConfig';
  RETURN;
END;

INSERT dbo.AGMonitorRunHistory
(
  ServerName,
  HostName,
  RunStartTime
)
VALUES
(
  @ServerName,
  @HostName,
  @RunStart
);

SET @RunID = SCOPE_IDENTITY();

CREATE TABLE #Findings
(
  FindingOrder  int IDENTITY(1,1) NOT NULL,
  Severity      varchar(10)  NOT NULL,
  Category      varchar(60)  NOT NULL,
  AGName        sysname      NULL,
  DatabaseName  sysname      NULL,
  ReplicaName   sysname      NULL,
  Observation    nvarchar(2048) NOT NULL,
  Recommendation nvarchar(2048) NULL
);

```

```
BEGIN TRY
```

```
/*-----
A. BASIC AG EXISTENCE
-----*/
```

```
IF NOT EXISTS (SELECT 1 FROM sys.availability_groups)
BEGIN
```

```
    INSERT #Findings
    (
        Severity, Category, Observation, Recommendation
    )
    VALUES
    (
        'WARNING',
        'Availability Group',
        N'Always On is enabled on the SQL Server instance, but no availability group metadata is currently visible.',
        N'Confirm the expected AG configuration, WSFC state, and whether this instance should host an availability replica.'
    );
END;
```

```
/*-----
B. DETERMINE WHETHER THIS INSTANCE CURRENTLY HOSTS A PRIMARY
-----*/
```

```
IF EXISTS
(
    SELECT 1
    FROM sys.dm_hadr_availability_replica_states
    WHERE is_local = 1
    AND role_desc = 'PRIMARY'
)
SET @HasPrimary = 1;
```

```
/*-----
C. REPLICA STATE
-----*/
```

```
SELECT
    ag.group_id,
    ag.name           AS AGName,
    ag.is_distributed,
    ar.replica_id,
    ar.replica_server_name,
    ar.endpoint_url,
    ar.availability_mode_desc,
    ar.failover_mode_desc,
    ars.is_local,
    ars.role_desc,
    ars.operational_state_desc,
    ars.connected_state_desc,
    ars.recovery_health_desc,
    ars.synchronization_health_desc,
    ars.last_connect_error_number,
    ars.last_connect_error_description,
    ars.last_connect_error_timestamp
INTO #Replica
FROM sys.availability_groups AS ag
INNER JOIN sys.availability_replicas AS ar
    ON ag.group_id = ar.group_id
LEFT JOIN sys.dm_hadr_availability_replica_states AS ars
```

```

ON ar.group_id = ars.group_id
AND ar.replica_id = ars.replica_id;

INSERT #Findings
SELECT
  'CRITICAL',
  'Replica Connectivity',
  AGName,
  NULL,
  replica_server_name,
  CONCAT(
    N'Replica is DISCONNECTED. Endpoint = ',
    COALESCE(endpoint_url, N'UNKNOWN'),
    N'. Last connection error = ',
    COALESCE(CONVERT(nvarchar(20), last_connect_error_number), N'UNKNOWN'),
    N' - ',
    COALESCE(last_connect_error_description, N'No description available'),
    N'.'
  ),
  N'Check SQL Server service, HADR endpoint, DNS, TCP connectivity, firewall rules, endpoint authentication/certificates, network path, and SQL Server error log.'
FROM #Replica
WHERE connected_state_desc = 'DISCONNECTED';

INSERT #Findings
SELECT
  'CRITICAL',
  'Replica Operational State',
  AGName,
  NULL,
  replica_server_name,
  CONCAT(N'Replica operational state is ', operational_state_desc, N'.').
  N'Check WSFC/quorum, SQL Server service state, cluster events, replica connectivity, and Windows/System logs.'
FROM #Replica
WHERE operational_state_desc IN ('FAILED', 'FAILED_NO_QUORUM', 'OFFLINE');

INSERT #Findings
SELECT
  CASE
    WHEN synchronization_health_desc = 'NOT_HEALTHY' THEN 'CRITICAL'
    ELSE 'WARNING'
  END,
  'Replica Synchronization Health',
  AGName,
  NULL,
  replica_server_name,
  CONCAT(N'Replica synchronization health is ', synchronization_health_desc, N'.').
  N'Check database synchronization state, suspended data movement, send/redo queues, replica connectivity, and HADR error log entries.'
FROM #Replica
WHERE synchronization_health_desc IN ('NOT_HEALTHY', 'PARTIALLY_HEALTHY');

INSERT #Findings
SELECT
  'WARNING',
  'Recent Replica Connection Error',
  AGName,
  NULL,
  replica_server_name,
  CONCAT(
    N'Error ', last_connect_error_number, N': ',
    COALESCE(last_connect_error_description, N'No description'),
    N' at '

```

```

        CONVERT(nvarchar(19), last_connect_error_timestamp, 120),
        N'.'
    ),
    N'Validate endpoint URL, DNS, firewall, TCP port, network stability, service account permissions/certificates, and remote SQL availability.'
FROM #Replica
WHERE ISNULL(last_connect_error_number, 0) <> 0
AND last_connect_error_timestamp >= DATEADD(HOUR, -@ErrorLookbackHours, @Now);

/*-----
D. DATABASE REPLICA STATE
-----*/

SELECT
    ag.name                                AS AGName,
    ag.is_distributed,
    DB_NAME(drs.database_id)              AS DatabaseName,
    ar.replica_server_name,
    drs.is_local,
    ars.role_desc,
    ar.availability_mode_desc,
    ars.connected_state_desc,
    drs.synchronization_state_desc,
    drs.synchronization_health_desc,
    drs.database_state_desc,
    drs.is_suspended,
    drs.suspend_reason_desc,
    drs.log_send_queue_size,
    drs.log_send_rate,
    CAST(
        CASE
            WHEN drs.log_send_queue_size > 0
            AND drs.log_send_rate > 0
            THEN drs.log_send_queue_size * 1.0 / drs.log_send_rate
        END
        AS decimal(18,2)
    ) AS EstimatedSendDelaySeconds,
    drs.redo_queue_size,
    drs.redo_rate,
    CAST(
        CASE
            WHEN drs.redo_queue_size > 0
            AND drs.redo_rate > 0
            THEN drs.redo_queue_size * 1.0 / drs.redo_rate
        END
        AS decimal(18,2)
    ) AS EstimatedRedoDelaySeconds,
    drs.secondary_lag_seconds,
    drs.last_sent_time,
    drs.last_received_time,
    drs.last_harden_time,
    drs.last_redone_time,
    drs.last_commit_time
INTO #DBState
FROM sys.dm_hadr_database_replica_states AS drs
INNER JOIN sys.availability_groups AS ag
    ON drs.group_id = ag.group_id
INNER JOIN sys.availability_replicas AS ar
    ON drs.group_id = ar.group_id
    AND drs.replica_id = ar.replica_id
LEFT JOIN sys.dm_hadr_availability_replica_states AS ars
    ON drs.group_id = ars.group_id

```

```

AND drs.replica_id = ars.replica_id;

INSERT dbo.AGMonitorDatabaseSnapshot
(
    RunID, CaptureTime, AGName, IsDistributed, DatabaseName,
    ReplicaName, IsLocal, RoleDesc, AvailabilityModeDesc,
    ConnectedStateDesc, SynchronizationStateDesc,
    SynchronizationHealthDesc, DatabaseStateDesc, IsSuspended,
    SuspendReasonDesc, LogSendQueueKB, LogSendRateKBps,
    EstimatedSendDelaySeconds, RedoQueueKB, RedoRateKBps,
    EstimatedRedoDelaySeconds, SecondaryLagSeconds,
    LastSentTime, LastReceivedTime, LastHardenedTime,
    LastRedoneTime, LastCommitTime
)
SELECT
    @RunID, @Now, AGName, is_distributed, DatabaseName,
    replica_server_name, is_local, role_desc, availability_mode_desc,
    connected_state_desc, synchronization_state_desc,
    synchronization_health_desc, database_state_desc, is_suspended,
    suspend_reason_desc, log_send_queue_size, log_send_rate,
    EstimatedSendDelaySeconds, redo_queue_size, redo_rate,
    EstimatedRedoDelaySeconds, secondary_lag_seconds,
    last_sent_time, last_received_time, last_hardened_time,
    last_redone_time, last_commit_time
FROM #DBState;

/* NOT SYNCHRONIZING */

INSERT #Findings
SELECT
    'CRITICAL',
    'Database Synchronization',
    AGName,
    DatabaseName,
    replica_server_name,
    CONCAT(
        N'Database synchronization state is ',
        COALESCE(synchronization_state_desc, N'UNKNOWN'),
        N'; synchronization health is ',
        COALESCE(synchronization_health_desc, N'UNKNOWN'),
        N'.'
    ),
    N'Check replica connectivity, HADR endpoint, transaction log transport, suspended data movement, storage health, and SQL Server error log.'
FROM #DBState
WHERE synchronization_state_desc = 'NOT SYNCHRONIZING';

/* SUSPENDED */

INSERT #Findings
SELECT
    'CRITICAL',
    'Data Movement Suspended',
    AGName,
    DatabaseName,
    replica_server_name,
    CONCAT(
        N'Data movement is suspended. Suspend reason = ',
        COALESCE(suspend_reason_desc, N'UNKNOWN'),
        N'.'
    ),
    N'Investigate the suspend reason and SQL Server error log first. Resume HADR only after the underlying cause is understood and corrected.'

```

```

FROM #DBState
WHERE is_suspended = 1;

/* BAD DATABASE STATE */

INSERT #Findings
SELECT
    'CRITICAL',
    'Database State',
    AGName,
    DatabaseName,
    replica_server_name,
    CONCAT(N'Availability database state is ', database_state_desc, N'.'),
    N'Investigate database recovery, storage/I/O, SQL Server error log, and availability-group health immediately.'
FROM #DBState
WHERE database_state_desc IN
(
    'RECOVERY_PENDING', 'SUSPECT', 'EMERGENCY', 'OFFLINE'
);

/* SEND QUEUE SIZE */

INSERT #Findings
SELECT
    CASE
        WHEN log_send_queue_size >= @SendQueueCriticalMB * 1024 THEN 'CRITICAL'
        ELSE 'WARNING'
    END,
    'Log Send Queue',
    AGName,
    DatabaseName,
    replica_server_name,
    CONCAT(
        N'Log send queue = ',
        CONVERT(nvarchar(30), CAST(log_send_queue_size / 1024.0 AS decimal(18,2))),
        N' MB; send rate = ',
        COALESCE(CONVERT(nvarchar(30), log_send_rate), N'NULL'),
        N' KB/sec; estimated drain time = ',
        COALESCE(CONVERT(nvarchar(30), EstimatedSendDelaySeconds), N'N/A'),
        N' seconds.'
    ),
    N'Check log generation rate, network throughput/latency, remote replica hardening, storage latency, CPU pressure, and replica connectivity.'
FROM #DBState
WHERE log_send_queue_size >= @SendQueueWarningMB * 1024;

/* SEND DELAY */

INSERT #Findings
SELECT
    CASE
        WHEN EstimatedSendDelaySeconds >= @DelayCriticalSec THEN 'CRITICAL'
        ELSE 'WARNING'
    END,
    'Estimated Send Delay',
    AGName,
    DatabaseName,
    replica_server_name,
    CONCAT(
        N'Estimated send queue drain time = ',
        CONVERT(nvarchar(30), EstimatedSendDelaySeconds),
        N' seconds.'
    )

```

```

),
N'Investigate transaction-log generation versus send throughput. Queue size alone is not enough; trend queue size and send rate together.'
FROM #DBState
WHERE EstimatedSendDelaySeconds >= @DelayWarningSec;

/* SEND STALL */

INSERT #Findings
SELECT
    'CRITICAL',
    'Log Transport Stalled',
    AGName,
    DatabaseName,
    replica_server_name,
    CONCAT(
        N'Log send queue contains ',
        CONVERT(nvarchar(30), CAST(log_send_queue_size / 1024.0 AS decimal(18,2))),
        N' MB while log_send_rate is 0 KB/sec.'
    ),
    N'Check replica connectivity, HADR endpoint/network path, remote SQL service, flow control, storage, and HADR errors.'
FROM #DBState
WHERE log_send_queue_size > 0
    AND ISNULL(log_send_rate, 0) = 0
    AND synchronization_state_desc = 'NOT SYNCHRONIZING';

/* REDO QUEUE SIZE */

INSERT #Findings
SELECT
    CASE
        WHEN redo_queue_size >= @RedoQueueCriticalMB * 1024 THEN 'CRITICAL'
        ELSE 'WARNING'
    END,
    'Redo Queue',
    AGName,
    DatabaseName,
    replica_server_name,
    CONCAT(
        N'Redo queue = ',
        CONVERT(nvarchar(30), CAST(redo_queue_size / 1024.0 AS decimal(18,2))),
        N' MB; redo rate = ',
        COALESCE(CONVERT(nvarchar(30), redo_rate), N'NULL'),
        N' KB/sec; estimated drain time = ',
        COALESCE(CONVERT(nvarchar(30), EstimatedRedoDelaySeconds), N'N/A'),
        N' seconds.'
    ),
    N'Check secondary replica CPU, storage latency/throughput, read workload, blocking, and redo performance.'
FROM #DBState
WHERE redo_queue_size >= @RedoQueueWarningMB * 1024;

/* REDO DELAY */

INSERT #Findings
SELECT
    CASE
        WHEN EstimatedRedoDelaySeconds >= @DelayCriticalSec THEN 'CRITICAL'
        ELSE 'WARNING'
    END,
    'Estimated Redo Delay',
    AGName,
    DatabaseName,

```

```

    replica_server_name,
    CONCAT(
        N'Estimated redo queue drain time = ',
        CONVERT(nvarchar(30), EstimatedRedoDelaySeconds),
        N' seconds.'
    ),
    N'Investigate secondary CPU, data-file I/O latency, storage throughput, read-only workload, and redo blocking/pressure.'
FROM #DBState
WHERE EstimatedRedoDelaySeconds >= @DelayWarningSec;

/* SECONDARY LAG */

INSERT #Findings
SELECT
    CASE
        WHEN secondary_lag_seconds >= @LagCriticalSec THEN 'CRITICAL'
        ELSE 'WARNING'
    END,
    'Secondary Lag',
    AGName,
    DatabaseName,
    replica_server_name,
    CONCAT(
        N'Secondary lag = ',
        CONVERT(nvarchar(30), secondary_lag_seconds),
        N' seconds.'
    ),
    N'Check log transport, network latency, log hardening, redo throughput, secondary workload, and storage performance.'
FROM #DBState
WHERE secondary_lag_seconds >= @LagWarningSec;

/*-----
E. HADR ENDPOINT
-----*/

SELECT
    dme.name,
    dme.state_desc,
    dme.connection_auth_desc,
    dme.is_encryption_enabled,
    te.port
INTO #Endpoint
FROM sys.database_mirroring_endpoints AS dme
LEFT JOIN sys.tcp_endpoints AS te
    ON dme.endpoint_id = te.endpoint_id;

IF NOT EXISTS (SELECT 1 FROM #Endpoint)
BEGIN
    INSERT #Findings
    (
        Severity, Category, Observation, Recommendation
    )
VALUES
    (
        'CRITICAL',
        'HADR Endpoint',
        N'No database mirroring/HADR endpoint was found on this Always On enabled instance.',
        N'Validate Always On endpoint configuration and the expected AG replica configuration.'
    );
END;

```

```

INSERT #Findings
SELECT
    'CRITICAL',
    'HADR Endpoint',
    NULL,
    NULL,
    @ServerName,
    CONCAT(
        N'Endpoint ', name,
        N' is ', state_desc,
        N'; TCP port = ',
        COALESCE(CONVERT(nvarchar(10), port), N'UNKNOWN'),
        N'.'
    ),
    N'The HADR endpoint should normally be STARTED. Check endpoint configuration, endpoint permissions/authentication, port binding, firewall, and network reachability.'
FROM #Endpoint
WHERE state_desc <> 'STARTED';

```

```

/*-----
F. LOCAL DISK SPACE FOR SQL DATABASE VOLUMES
-----*/

```

```

SELECT
    vs.volume_mount_point,
    MAX(vs.total_bytes) AS total_bytes,
    MAX(vs.available_bytes) AS available_bytes,
    CAST(
        MAX(vs.available_bytes) * 100.0 /
        NULLIF(MAX(vs.total_bytes), 0)
        AS decimal(10,2)
    ) AS FreePct
INTO #Disk
FROM sys.master_files AS mf
CROSS APPLY sys.dm_os_volume_stats(mf.database_id, mf.file_id) AS vs
GROUP BY vs.volume_mount_point;

```

```

INSERT #Findings
SELECT
    CASE
        WHEN FreePct <= @DiskCriticalPct THEN 'CRITICAL'
        ELSE 'WARNING'
    END,
    'Disk Space',
    NULL,
    NULL,
    @ServerName,
    CONCAT(
        N'Volume ', volume_mount_point,
        N' has ', CONVERT(nvarchar(30), FreePct),
        N'% free space (',
        CONVERT(nvarchar(30), CAST(available_bytes / 1073741824.0 AS decimal(18,2))),
        N' GB free of ',
        CONVERT(nvarchar(30), CAST(total_bytes / 1073741824.0 AS decimal(18,2))),
        N' GB).'
    ),
    N'Free space or increase storage capacity before data/log growth, backup activity, or SQL Server operations are affected.'
FROM #Disk
WHERE FreePct <= @DiskWarningPct;

```

```

/*-----
G. LOG REUSE WAIT

```

```

-----*/

INSERT #Findings
SELECT
    'WARNING',
    'Transaction Log Reuse',
    ag.name,
    d.name,
    @ServerName,
    N'log_reuse_wait_desc = AVAILABILITY_REPLICA.',
    N'Check disconnected/lagging replicas, log-send queues, suspended movement, and replica health before the transaction log grows excessively.'
FROM sys.databases AS d
INNER JOIN sys.availability_databases_cluster AS adc
    ON d.name = adc.database_name
INNER JOIN sys.availability_groups AS ag
    ON adc.group_id = ag.group_id
WHERE d.log_reuse_wait_desc = 'AVAILABILITY_REPLICA';

/*-----
H. RECENT SQL SERVER ERRORLOG - HADR/AVAILABILITY RELATED
-----*/

CREATE TABLE #ErrorLogRaw
(
    LogDate    datetime,
    ProcessInfo nvarchar(50),
    [Text]     nvarchar(max)
);

BEGIN TRY
    INSERT #ErrorLogRaw
    EXEC master.sys.sp_readerrorlog 0, 1, N'HADR';
END TRY
BEGIN CATCH
    INSERT #Findings
    (
        Severity, Category, Observation, Recommendation
    )
    VALUES
    (
        'WARNING',
        'Error Log Monitoring',
        CONCAT(N'Unable to read HADR entries from SQL Server ERRORLOG: ', ERROR_MESSAGE()),
        N'On SQL Server 2022 and later, verify VIEW ANY ERROR LOG or VIEW SERVER PERFORMANCE STATE as appropriate for the execution context.'
    );
END CATCH;

BEGIN TRY
    INSERT #ErrorLogRaw
    EXEC master.sys.sp_readerrorlog 0, 1, N'availability';
END TRY
BEGIN CATCH
    -- Avoid duplicating permission finding if first call already reported it.
END CATCH;

;WITH E AS
(
    SELECT DISTINCT
        LogDate,
        ProcessInfo,
        [Text]

```

```

FROM #ErrorLogRaw
WHERE LogDate >= DATEADD(HOUR, -@ErrorLookbackHours, GETDATE())
AND
(
    [Text] LIKE N'%error%'
    OR [Text] LIKE N'%fail%'
    OR [Text] LIKE N'%disconnect%'
    OR [Text] LIKE N'%timeout%'
    OR [Text] LIKE N'%suspend%'
    OR [Text] LIKE N'%lease%'
    OR [Text] LIKE N'%not synchron%'
    OR [Text] LIKE N'%terminated%'
)
)
INSERT #Findings
SELECT TOP (25)
CASE
    WHEN [Text] LIKE N'%fail%'
    OR [Text] LIKE N'%error%'
    OR [Text] LIKE N'%lease%'
    OR [Text] LIKE N'%terminated%'
    THEN 'CRITICAL'
    ELSE 'WARNING'
END,
'Recent HADR Error Log',
NULL,
NULL,
@ServerName,
CONCAT(
    CONVERT(nvarchar(19), LogDate, 120),
    N' | ', ProcessInfo,
    N' | ', LEFT([Text], 1700)
),
N'Review the surrounding SQL Server ERRORLOG entries and correlate them with replica connectivity, WSFC, network, endpoint, storage, and database-state findings.'
FROM E
ORDER BY LogDate DESC;

/*-----
I. HEALTHY MARKER
-----*/

IF NOT EXISTS (SELECT 1 FROM #Findings)
BEGIN
    INSERT #Findings
    (
        Severity, Category, Observation, Recommendation
    )
VALUES
    (
        'HEALTHY',
        'Overall Health',
        N'No warning or critical conditions were detected by the configured SQL-visible Always On checks.',
        N'Continue normal monitoring. Review historical queue and lag trends to tune thresholds for this workload.'
    );
END;

/*-----
J. PERSIST FINDINGS
-----*/

INSERT dbo.AGMonitorFindingHistory

```

```

(
    RunID,
    CaptureTime,
    Severity,
    Category,
    AGName,
    DatabaseName,
    ReplicaName,
    Observation,
    Recommendation
)
)
SELECT
    @RunID,
    @Now,
    Severity,
    Category,
    AGName,
    DatabaseName,
    ReplicaName,
    Observation,
    Recommendation
FROM #Findings;

SELECT
    @CriticalCount = SUM(CASE WHEN Severity = 'CRITICAL' THEN 1 ELSE 0 END),
    @WarningCount = SUM(CASE WHEN Severity = 'WARNING' THEN 1 ELSE 0 END),
    @HealthyCount = SUM(CASE WHEN Severity = 'HEALTHY' THEN 1 ELSE 0 END)
FROM #Findings;

SET @CriticalCount = ISNULL(@CriticalCount, 0);
SET @WarningCount = ISNULL(@WarningCount, 0);
SET @HealthyCount = ISNULL(@HealthyCount, 0);

SET @OverallStatus =
    CASE
        WHEN @CriticalCount > 0 THEN 'CRITICAL'
        WHEN @WarningCount > 0 THEN 'WARNING'
        ELSE 'HEALTHY'
    END;

/*-----
K. SHOULD THIS REPLICA SEND EMAIL?
-----*/

SET @ShouldSend =
    CASE
        WHEN @ForceEmail = 1 THEN 1
        WHEN @HasPrimary = 1 THEN 1
        WHEN @SendHealthyFromSecondary = 1 THEN 1
        WHEN @OverallStatus IN ('WARNING','CRITICAL') THEN 1
        ELSE 0
    END;

/*-----
L. HTML EMAIL
-----*/

SET @Subject =
    CONCAT(
        N'[AG ', @OverallStatus, N'] ',
        @ServerName,

```

```

        N' | 2-Hour Always On Health Check'
    );

    SET @Body =
    N'<!DOCTYPE html>
    <html>
    <head>
    <meta charset="utf-8">
    <style>
    body{font-family:Segoe UI,Arial,sans-serif;font-size:13px;color:#222}
    h2,h3{color:#17365D}
    table{border-collapse:collapse;width:100%;margin-bottom:18px}
    th{background:#D9EAF7;text-align:left}
    th,td{border:1px solid #B7C9D6;padding:6px;vertical-align:top}
    .status-HEALTHY{background:#DFF0D8;font-weight:bold}
    .status-WARNING{background:#FFF3CD;font-weight:bold}
    .status-CRITICAL{background:#F8D7DA;font-weight:bold}
    .small{font-size:11px;color:#555}
    </style>
    </head>
    <body>';

    SET @Body +=
    N'<h2>SQL Server Always On Health Report</h2>' +
    N'<table>' +
    N'<tr><th>SQL Server</th><td>' + dbo.ufn_AGMonitor_HtmlEncode(@ServerName) + N'</td></tr>' +
    N'<tr><th>Host</th><td>' + dbo.ufn_AGMonitor_HtmlEncode(@HostName) + N'</td></tr>' +
    N'<tr><th>Report Time</th><td>' + CONVERT(nvarchar(19), @Now, 120) + N' (server local time)</td></tr>' +
    N'<tr><th>Hosts a Primary Replica</th><td>' + CASE WHEN @HasPrimary = 1 THEN N'YES' ELSE N'NO' END + N'</td></tr>' +
    N'<tr><th>Overall Status</th><td class="status-' + @OverallStatus + N'">' + @OverallStatus + N'</td></tr>' +
    N'<tr><th>Critical / Warning</th><td>' + CONVERT(nvarchar(12), @CriticalCount) + N' / ' + CONVERT(nvarchar(12), @WarningCount) + N'</td></tr>' +
    N'</table>';

    /* Findings table */

    SELECT @Rows =
    STRING_AGG(
        CAST(
            N'<tr>' +
            N'<td class="status-' + Severity + N'">' + Severity + N'</td>' +
            N'<td>' + dbo.ufn_AGMonitor_HtmlEncode(Category) + N'</td>' +
            N'<td>' + dbo.ufn_AGMonitor_HtmlEncode(ISNULL(AGName,N'')) + N'</td>' +
            N'<td>' + dbo.ufn_AGMonitor_HtmlEncode(ISNULL(DatabaseName,N'')) + N'</td>' +
            N'<td>' + dbo.ufn_AGMonitor_HtmlEncode(ISNULL(ReplicaName,N'')) + N'</td>' +
            N'<td>' + dbo.ufn_AGMonitor_HtmlEncode(Observation) + N'</td>' +
            N'<td>' + dbo.ufn_AGMonitor_HtmlEncode(ISNULL(Recommendation,N'')) + N'</td>' +
            N'</tr>'
            AS nvarchar(max)
        ),
        N''
    )
    FROM #Findings;

    SET @Body +=
    N'<h3>Detected Health Findings</h3>' +
    N'<table><tr>' +
    N'<th>Severity</th><th>Category</th><th>AG</th><th>Database</th><th>Replica</th><th>Observation</th><th>Recommendation</th>' +
    N'</tr>' + ISNULL(@Rows,N'') + N'</table>';

    /* Replica details */

```

```

SELECT @Rows =
STRING_AGG(
  CAST(
    N'<tr>' +
    N'<td>' + dbo.ufn_AGMonitor_HtmlEncode(AGName) + N'</td>' +
    N'<td>' + CASE WHEN is_distributed = 1 THEN N'YES' ELSE N'NO' END + N'</td>' +
    N'<td>' + dbo.ufn_AGMonitor_HtmlEncode(replica_server_name) + N'</td>' +
    N'<td>' + dbo.ufn_AGMonitor_HtmlEncode(ISNULL(role_desc,N'Not visible from this replica')) + N'</td>' +
    N'<td>' + dbo.ufn_AGMonitor_HtmlEncode(ISNULL(availability_mode_desc,N'')) + N'</td>' +
    N'<td>' + dbo.ufn_AGMonitor_HtmlEncode(ISNULL(connected_state_desc,N'Not visible')) + N'</td>' +
    N'<td>' + dbo.ufn_AGMonitor_HtmlEncode(ISNULL(synchronization_health_desc,N'Not visible')) + N'</td>' +
    N'<td>' + dbo.ufn_AGMonitor_HtmlEncode(ISNULL(endpoint_url,N'')) + N'</td>' +
    N'</tr>'
    AS nvarchar(max)
  ),
  N''
)
FROM #Replica;

SET @Body +=
N'<h3>Availability Replica Summary</h3>' +
N'<table><tr>' +
N'<th>AG</th><th>Distributed</th><th>Replica</th><th>Role</th><th>Commit Mode</th><th>Connected</th><th>Sync Health</th><th>Endpoint</th>' +
N'</tr>' + ISNULL(@Rows,N'') + N'</table>';

/* Database details */

SELECT @Rows =
STRING_AGG(
  CAST(
    N'<tr>' +
    N'<td>' + dbo.ufn_AGMonitor_HtmlEncode(AGName) + N'</td>' +
    N'<td>' + dbo.ufn_AGMonitor_HtmlEncode(DatabaseName) + N'</td>' +
    N'<td>' + dbo.ufn_AGMonitor_HtmlEncode(replica_server_name) + N'</td>' +
    N'<td>' + dbo.ufn_AGMonitor_HtmlEncode(ISNULL(role_desc,N'')) + N'</td>' +
    N'<td>' + dbo.ufn_AGMonitor_HtmlEncode(ISNULL(synchronization_state_desc,N'')) + N'</td>' +
    N'<td>' + dbo.ufn_AGMonitor_HtmlEncode(ISNULL(synchronization_health_desc,N'')) + N'</td>' +
    N'<td>' + CASE WHEN is_suspended = 1 THEN N'YES' ELSE N'NO' END + N'</td>' +
    N'<td>' + COALESCE(CONVERT(nvarchar(30), CAST(log_send_queue_size/1024.0 AS decimal(18,2))),N'') + N'</td>' +
    N'<td>' + COALESCE(CONVERT(nvarchar(30), log_send_rate),N'') + N'</td>' +
    N'<td>' + COALESCE(CONVERT(nvarchar(30), EstimatedSendDelaySeconds),N'') + N'</td>' +
    N'<td>' + COALESCE(CONVERT(nvarchar(30), CAST(redo_queue_size/1024.0 AS decimal(18,2))),N'') + N'</td>' +
    N'<td>' + COALESCE(CONVERT(nvarchar(30), redo_rate),N'') + N'</td>' +
    N'<td>' + COALESCE(CONVERT(nvarchar(30), EstimatedRedoDelaySeconds),N'') + N'</td>' +
    N'<td>' + COALESCE(CONVERT(nvarchar(30), secondary_lag_seconds),N'') + N'</td>' +
    N'</tr>'
    AS nvarchar(max)
  ),
  N''
)
FROM #DBState;

SET @Body +=
N'<h3>Availability Database Details</h3>' +
N'<table><tr>' +
N'<th>AG</th><th>Database</th><th>Replica</th><th>Role</th><th>Sync State</th><th>Sync Health</th><th>Suspended</th>' +
N'<th>Send Queue MB</th><th>Send KB/s</th><th>Est. Send sec</th>' +
N'<th>Redo Queue MB</th><th>Redo KB/s</th><th>Est. Redo sec</th><th>Lag sec</th>' +
N'</tr>' + ISNULL(@Rows,N'') + N'</table>';

/* Disk */

```

```

SELECT @Rows =
  STRING_AGG(
    CAST(
      N'<tr><td>' + dbo.ufn_AGMonitor_HtmlEncode(volume_mount_point) + N'</td>' +
      N'<td>' + CONVERT(nvarchar(30), CAST(total_bytes/1073741824.0 AS decimal(18,2))) + N'</td>' +
      N'<td>' + CONVERT(nvarchar(30), CAST(available_bytes/1073741824.0 AS decimal(18,2))) + N'</td>' +
      N'<td>' + CONVERT(nvarchar(30), FreePct) + N'</td></tr>'
      AS nvarchar(max)
    ),
    N''
  )
FROM #Disk;

SET @Body +=
  N'<h3>SQL Database Volume Space</h3>' +
  N'<table><tr><th>Volume</th><th>Total GB</th><th>Free GB</th><th>Free %</th></tr>' +
  ISNULL(@Rows,N'') + N'</table>';

SET @Body +=
  N'<p class="small">Note: SYNCHRONIZING can be the expected healthy state for an asynchronous-commit secondary. The monitor therefore evaluates synchronization health, role/commit mode, queues, lag, and
related symptoms rather than treating every SYNCHRONIZING state as a failure.</p>' +
  N'<p class="small">Run ID: ' + CONVERT(nvarchar(30), @RunID) + N'</p>' +
  N'</body></html>';

/*-----
M. SEND MAIL
-----*/

IF @ShouldSend = 1
BEGIN
  UPDATE dbo.AGMonitorRunHistory
  SET EmailAttempted = 1
  WHERE RunID = @RunID;

  BEGIN TRY
    EXEC msdb.dbo.sp_send_dbmail
      @profile_name = @MailProfile,
      @recipients   = @Recipients,
      @subject      = @Subject,
      @body         = @Body,
      @body_format  = 'HTML';

    SET @EmailQueued = 1;
  END TRY
  BEGIN CATCH
    UPDATE dbo.AGMonitorRunHistory
    SET EmailError = ERROR_MESSAGE()
    WHERE RunID = @RunID;
  END CATCH;
END;

/*-----
N. RETENTION
-----*/

DELETE dbo.AGMonitorFindingHistory
WHERE CaptureTime < DATEADD(DAY, -@RetentionDays, @Now);

DELETE dbo.AGMonitorDatabaseSnapshot
WHERE CaptureTime < DATEADD(DAY, -@RetentionDays, @Now);

```

```
DELETE dbo.AGMonitorRunHistory
WHERE RunStartTime < DATEADD(DAY, -@RetentionDays, @Now)
AND RunID <> @RunID;
```

```
UPDATE dbo.AGMonitorRunHistory
SET
    RunEndTime    = SYSDATETIME(),
    OverallStatus = @OverallStatus,
    CriticalCount  = @CriticalCount,
    WarningCount  = @WarningCount,
    HealthyCount  = @HealthyCount,
    EmailQueued   = @EmailQueued
WHERE RunID = @RunID;
```

```
IF @Debug = 1
BEGIN
    SELECT
        @RunID AS RunID,
        @OverallStatus AS OverallStatus,
        @HasPrimary AS HostsPrimary,
        @ShouldSend AS ShouldSendEmail,
        @EmailQueued AS EmailQueued,
        @CriticalCount AS CriticalCount,
        @WarningCount AS WarningCount;
```

```
SELECT *
FROM #Findings
ORDER BY
    CASE Severity
        WHEN 'CRITICAL' THEN 1
        WHEN 'WARNING' THEN 2
        ELSE 3
    END,
    FindingOrder;
```

```
SELECT *
FROM #Replica
ORDER BY AGName, replica_server_name;
```

```
SELECT *
FROM #DBState
ORDER BY AGName, DatabaseName, replica_server_name;
END;
```

```
END TRY
BEGIN CATCH
    DECLARE @ProcedureError nvarchar(4000) =
        CONCAT(
            N'Error ', ERROR_NUMBER(),
            N', line ', ERROR_LINE(),
            N': ', ERROR_MESSAGE()
        );
```

```
UPDATE dbo.AGMonitorRunHistory
SET
    RunEndTime    = SYSDATETIME(),
    OverallStatus = 'CRITICAL',
    ProcedureError = @ProcedureError
WHERE RunID = @RunID;
```

```

/* Best-effort monitor failure email */
BEGIN TRY
    EXEC msdb.dbo.sp_send_dbmail
        @profile_name = @MailProfile,
        @recipients = @Recipients,
        @subject = N'[AG MONITOR FAILURE] SQL Server Always On Health Check',
        @body = @ProcedureError,
        @body_format = 'TEXT';
END TRY
BEGIN CATCH
    -- Do not mask the original procedure exception.
END CATCH;

    THROW;
END CATCH;
END;
GO

/*=====
8. REPORTING / TROUBLESHOOTING VIEWS
=====*/

CREATE OR ALTER VIEW dbo.vw_AGMonitorLatestRuns
AS
SELECT TOP (100)
    RunID,
    ServerName,
    HostName,
    RunStartTime,
    RunEndTime,
    OverallStatus,
    CriticalCount,
    WarningCount,
    HealthyCount,
    EmailAttempted,
    EmailQueued,
    EmailError,
    ProcedureError
FROM dbo.AGMonitorRunHistory
ORDER BY RunID DESC;
GO

CREATE OR ALTER VIEW dbo.vw_AGMonitorRecentProblems
AS
SELECT
    f.FindingID,
    f.RunID,
    r.ServerName,
    f.CaptureTime,
    f.Severity,
    f.Category,
    f.AGName,
    f.DatabaseName,
    f.ReplicaName,
    f.Observation,
    f.Recommendation
FROM dbo.AGMonitorFindingHistory AS f
INNER JOIN dbo.AGMonitorRunHistory AS r
    ON f.RunID = r.RunID
WHERE f.Severity IN ('WARNING','CRITICAL');
GO

```

```

/*=====
9. SQL SERVER AGENT JOB - EVERY TWO HOURS
=====*/

USE msdb;
GO

DECLARE
    @JobName sysname = N'DBA - Always On Health Alert - Every 2 Hours',
    @ScheduleName sysname = N'DBA - Always On Health - Every 2 Hours';

IF EXISTS (SELECT 1 FROM dbo.sysjobs WHERE name = @JobName)
BEGIN
    EXEC dbo.sp_delete_job
        @job_name = @JobName,
        @delete_unused_schedule = 1;
END;

EXEC dbo.sp_add_job
    @job_name = @JobName,
    @enabled = 1,
    @description = N'Runs the DBA_Admin Always On health monitor every two hours. Primary sends heartbeat; secondary sends fail-safe warning/critical notifications by default.';

EXEC dbo.sp_add_jobstep
    @job_name = @JobName,
    @step_name = N'Collect AG Health and Send Notification',
    @subsystem = N'TSQL',
    @database_name = N'DBA_Admin',
    @command = N'EXEC dbo.usp_AG_HealthCheck_Email @ForceEmail = 0, @Debug = 0;',
    @retry_attempts = 2,
    @retry_interval = 5,
    @on_success_action = 1,
    @on_fail_action = 2;

EXEC dbo.sp_add_jobschedule
    @job_name = @JobName,
    @name = @ScheduleName,
    @enabled = 1,
    @freq_type = 4,      -- Daily
    @freq_interval = 1,  -- Every day
    @freq_subday_type = 8, -- Hours
    @freq_subday_interval = 2, -- Every 2 hours
    @active_start_time = 000000; -- 00:00 server local time

EXEC dbo.sp_add_jobserver
    @job_name = @JobName;
GO

/*=====
10. POST-DEPLOYMENT TESTS
=====*/

USE DBA_Admin;
GO

PRINT '=====';
PRINT 'Always On monitor deployment completed.';
PRINT '1. Update dbo.AGMonitorConfig with the correct mail profile.';
PRINT '2. Update dbo.AGMonitorConfig with the correct recipients.';
PRINT '3. Run the forced email test below.';

```

```

PRINT '=====';
GO

/* Force a test email from THIS replica, even if it is secondary */
-- EXEC dbo.usp_AG_HealthCheck_Email @ForceEmail = 1, @Debug = 1;
-- GO

/* Normal production behavior */
-- EXEC dbo.usp_AG_HealthCheck_Email @ForceEmail = 0, @Debug = 1;
-- GO

/* Recent runs */
-- SELECT * FROM dbo.vw_AGMonitorLatestRuns;
-- GO

/* Recent problems */
-- SELECT TOP (100) *
-- FROM dbo.vw_AGMonitorRecentProblems
-- ORDER BY CaptureTime DESC;
-- GO

/* Queue/lag trend for the last 24 hours */
-- SELECT
--     CaptureTime,
--     AGName,
--     DatabaseName,
--     ReplicaName,
--     LogSendQueueKB / 1024.0 AS LogSendQueueMB,
--     LogSendRateKBps,
--     EstimatedSendDelaySeconds,
--     RedoQueueKB / 1024.0 AS RedoQueueMB,
--     RedoRateKBps,
--     EstimatedRedoDelaySeconds,
--     SecondaryLagSeconds
-- FROM dbo.AGMonitorDatabaseSnapshot
-- WHERE CaptureTime >= DATEADD(HOUR, -24, SYSDATETIME())
-- ORDER BY CaptureTime DESC, AGName, DatabaseName, ReplicaName;
-- GO

```

## Appendix B - Production Deployment Checklist

| Check                                             | Status / Notes |
|---------------------------------------------------|----------------|
| Database Mail profile tested                      |                |
| Recipients confirmed                              |                |
| Permissions validated under SQL Agent context     |                |
| Script deployed on every AG replica               |                |
| Forced test email successful on every replica     |                |
| Primary heartbeat received                        |                |
| Secondary healthy suppression/behavior confirmed  |                |
| Warning/critical test performed in non-production |                |

| Check                                                | Status / Notes |
|------------------------------------------------------|----------------|
| History tables populated                             |                |
| 90-day retention approved                            |                |
| Thresholds aligned with RPO/RTO and workload         |                |
| WSFC/network/storage monitoring ownership documented |                |
| Runbook/RCA ownership assigned                       |                |